Learning the Video QC Project

A stage by stage guide for Haroon from business concepts to operating the system

14 October 2026

This guide explains what we built in your WSL repository, why each component exists, how to operate it, and what you can confidently tell your mentor. No computer science background is assumed. Read the story first, then the stages and technical explanations, then practise the commands. The pipeline processes the supplied recordings and the first local dashboard works. Independent real-data classification accuracy has not been established.

How to use this guide

Read sections 1 to 3 for the story and vocabulary. Sections 4 to 17 explain the build stages. Section 18 teaches report reading. Sections 19 to 22 contain commands and troubleshooting. Sections 23 to 25 prepare you for mentor questions. Sections 26 to 28 cover files, versions and remaining work.

1 The story to tell your mentor

We began with a laptop and a partly working setup. Postgres was running, but MinIO could not download its image. Before judging recordings, we needed somewhere reliable to run the software, preserve originals, remember processing steps, and save decisions.

Imagine setting up an audit department. The MCAP recordings are sealed evidence bundles. MinIO is the warehouse. Postgres is the official ledger. Temporal is the case coordinator who remembers what each auditor has completed. Worker pods are the auditors doing the checks. ClickHouse is the management reporting workbook. The dashboard is the front desk where you submit a case and inspect the evidence.

We first built the department. We repaired MinIO, added ClickHouse, installed Temporal using Postgres, and ran a small hello-world workflow. We automated startup and tested teardown and startup from a fresh clone with an empty cluster. This completed the infrastructure foundation.

When your mentor supplied 41 recordings, we learned their cameras, sensors, formats and timing. We wrote first impressions before building the automated checks. Then we added content fingerprints, protected uploaded originals, and made Temporal coordinate container, timing, image and cross-stream checks.

We tested duplicate uploads, deliberately damaged copies, workflow replay, a separately approved worker crash and a rebuild from raw. We chose and documented the quality policy. The final results were 29 pass, 1 fail and 11 needs_review. The failed recording had near-black images across all four cameras. The review cases had unchanged encoder readings that can mean legitimate stillness.

Finally we built a local dashboard. You can upload, search, filter, inspect findings, download reports and view sampled camera images. Full playback, saved human review decisions, richer progress and retry controls, and Kubernetes packaging for the dashboard remain future improvements.

A short version to say aloud

I built a local quality-control pipeline for robot recordings. It protects originals, checks structure, timing and content, and records explainable decisions. Kubernetes runs the services, Temporal coordinates recoverable work, Postgres stores official decisions, and ClickHouse stores analytical metrics. I verified all 41 recordings and tested duplicate handling and worker recovery. The dashboard helps me inspect the evidence. My thresholds are documented engineering choices, not a claim of perfect accuracy.

2 The goal and the business controls

The exercise teaches you to operate a local version of a data-processing system. The practical goal is to decide whether multi-camera robot recordings meet a declared quality standard and say exactly why. A file can open normally but contain missing cameras, dark images, timing gaps or invalid sensor values.

Business idea	Project equivalent
Original invoice or audit evidence	Immutable raw recording
Identity derived from actual evidence	SHA-256 digest
Official ledger	Postgres episodes and verdicts
Audit procedure and tolerances	Configured QC policy
Case coordinator and audit trail	Temporal workflow and history
Staff carrying out procedures	Worker activities
Management reporting	ClickHouse metrics and fleet queries
Finding supported by evidence	Reason code, stream, interval and metrics

Our main controls are immutable raw, content identity, rebuildable derived outputs, a fixed verdict vocabulary, fail-closed checks, repeatable submissions and one-command infrastructure startup. These prevent damaged evidence, double-counting and unjustified approval.

- pass: implemented required checks finished and found no violation of the current policy.

- fail: at least one failure rule triggered, or a required check could not complete.

- needs_review: evidence is ambiguous and needs context, such as encoder stillness or apparent freezing.

A successful workflow and a passing recording are different. Completed means the processing job finished. A completed job can correctly return fail. This is similar to an audit completing successfully and discovering an accounting issue.

3 Understand your laptop and the repository

Windows is the host operating system. WSL means Windows Subsystem for Linux. It lets us run Ubuntu and Linux tools alongside Windows. All project edits belong in Ubuntu at /home/haroon/video-qc. Windows sees the same folder through the path below; it is not another copy.

```
\\wsl$\Ubuntu\home\haroon\video-qc
```

The repo is the folder tracked by Git. Code, manifests, documentation and small evidence files belong there. Git commits are local snapshots. GitHub is the remote repository. Saving a file, committing it and pushing it are separate actions. Original MCAPs, large viewer files and caches are ignored rather than committed.

Docker builds and runs containers. An image is a packaged application template with its runtime and libraries. A container is a running instance. kind means Kubernetes in Docker; it creates local Kubernetes nodes as Docker containers.

Our cluster name is qc, kubectl context is kind-qc, and Kubernetes namespace is qc. A context selects a cluster; a namespace groups its resources. Temporal has its own namespace concept. Our Temporal workflows use default, which is separate from Kubernetes namespace qc.

The original files are in C:\Users\win 11\Downloads\data, read through /mnt/c/Users/win 11/Downloads/data in WSL. Ubuntu paths use forward slashes. The space in win 11 must remain quoted in commands.

```
wsl -d Ubuntu -- bash -lc "cd /home/haroon/video-qc && make status"
```

Read the command left to right: start Ubuntu through WSL, run Bash, change to the repository, then run the named Make target. The && means execute the next part only if the previous part succeeds.

Your rules kept work inside the repo and qc cluster scope, required pinned official/upstream software and small resources, protected originals, prohibited reading or printing credential values, and required approval for destructive steps. Prior approval to upload code overrode the initial no-push rule for the work you authorized.

4 Stage one repair MinIO and add ClickHouse

The starting MinIO pod had ImagePullBackOff. A pod is Kubernetes' unit for running containers. ImagePullBackOff means Kubernetes could not obtain an image and is delaying its next attempt. Describing the pod reveals its actual pull errors.

The attempted community MinIO image pulls failed, including 401 responses and missing tags. You approved building from official upstream source. We pinned the source release and checked its commit, built a local image, verified it locally and loaded it into kind. We did not silently substitute an unrelated object store.

MinIO is S3-compatible object storage. S3 compatibility means its API works with standard object-storage clients such as boto3. A bucket groups objects, and a key names an object. Prefixes such as raw/ look like folders but are part of the object key.

We added ClickHouse using its official pinned image. It is an analytical database for scanning numeric measurements and aggregating results. Later it answers which camera has the most estimated dropped frames across the mentor dataset.

Postgres, MinIO and ClickHouse use StatefulSets with persistent volume claims. A StatefulSet supplies stable identities and storage association. A PVC requests storage. Bound means Kubernetes assigned storage. It survives individual pod replacement in the existing cluster, but not deleting the local kind cluster.

Requests tell Kubernetes what resources to reserve. Limits bound a container's consumption. 100m CPU is one tenth of a core. Mi and Gi are binary memory units. Small settings keep the laptop usable; overly small memory limits can kill workloads.

Mentor takeaway: we diagnosed the image-distribution problem, used pinned upstream source, and gave stateful services storage appropriate for pod replacement. Persistent storage is not an independent backup.

5 Stage two install Temporal and run hello world

Temporal is a durable workflow engine. It stores the processing history so work can continue after a worker stops. It is like a case coordinator with an audit trail, rather than a script that forgets everything when its process dies.

We installed the official temporalio Helm chart, pinned to version 1.7.0. Helm is a package manager for Kubernetes. A chart contains resource templates; k8s/temporal-values.yaml supplies our settings. Temporal uses the existing Postgres, enables its web UI and has laptop-sized resource bounds.

Temporal persistence stores durable execution data. Visibility supports listing and searching executions. The frontend receives requests, history manages execution history, and matching connects work with workers. Temporal's internal worker component is different from our application QC workers.

We wrote a minimal Python hello workflow, an activity, a Dockerfile, a worker Deployment and a start script. We built and loaded the image into kind and ran it from a pod. It returned Hello, world!, showed Completed in Temporal's UI and passed history replay.

A workflow decides the sequence of work. An activity performs work that can read files, call services or decode data. A worker executes compatible tasks from a named task queue. The queue holds coordination work, not the gigabytes of recording data.

Workflow code must be deterministic: the same recorded history must reconstruct the same scheduling decisions. Real clocks, randomness, filesystem reads and network calls belong in activities or outside workflow code. Replay reuses recorded activity results instead of automatically repeating their external effects.

```
wsl -d Ubuntu -- bash -lc "cd /home/haroon/video-qc && make hello"
wsl -d Ubuntu -- bash -lc "cd /home/haroon/video-qc && make ui"
```

Keep make ui running and open http://localhost:8080. It creates a local port-forward to the Temporal UI. Closing the terminal closes that tunnel. The link itself does not start a service.

Mentor takeaway: the small hello example established the infrastructure before we trusted it with expensive recording checks.

6 Stage three automate startup and verify Day one

A Makefile names repeatable procedures. make up prepares pinned tools, creates qc if missing, applies services, builds and loads MinIO, installs Temporal, builds and deploys workers, initializes operational tables and waits for readiness. Repeating it normally reuses the cluster.

A manifest is YAML describing desired Kubernetes resources. Applying it asks Kubernetes to maintain that state. A Deployment maintains worker replicas; a StatefulSet manages stateful identities. Ready means the workload passed its readiness condition. Running alone is not sufficient proof.

make down is the teardown path. It asks you to type qc and deletes that cluster, including local database volumes, uploaded MinIO storage and Temporal history. It leaves your repo and Windows originals, but those are not backups of every in-cluster result.

With your approval we tested teardown, then a fresh shallow clone started with no qc cluster, database volumes or application Secrets. Ten service pods were Ready and two Temporal bootstrap jobs were Completed. Jobs should finish; long-running services should remain Ready.

This test reused Docker image and build caches. It proves empty-cluster startup under those conditions, not an uncached offline installation. Later tests used the populated cluster. After the QC deployment, the expanded stack had twelve Ready service pods and two Completed jobs.

- Ubuntu initially lacked Make; you approved installing GNU Make.

- Repeated registry checks exhausted anonymous Docker Hub allowance. make check-images became a separate audit target.

- Docker 29 image exports and kind 0.23 loading required a host-platform compatibility archive without changing image layers.

- Several WSL/Docker/node restarts and Forbidden responses interrupted checks. Final checks succeeded with an active WSL session; the restart cause remains unconfirmed. You confirmed you were not restarting them.

Mentor takeaway: automation is a repeatable operational procedure backed by readiness and a clearly scoped clean-clone test, not just a collection of commands.

7 Stage four learn the 41 recordings

The mentor supplied 41 MCAPs totaling about 5.1 GB. MCAP is binary, so strange characters in a text editor are expected. It contains timestamped messages from cameras and sensors; it is not simply one conventional video file.

A topic names a stream. A channel associates the topic with a schema and encoding. A schema describes the message fields. Encoding describes how those fields are represented. These files use protobuf messages; their compressed video payloads need a video decoder.

Chunks group stored messages. A chunk index helps readers locate data efficiently. A summary provides metadata. A readable file without a summary or required index can still violate our completeness contract. A truncated tail is different from random unreadable bytes.

We prepared a checksum-verified MCAP CLI and pinned Python inspection tools. CLI inspection reported container and stream metadata. Python decoded messages, examined timestamps and made contact sheets, grids of selected camera images.

The exercise originally asked for interactive per-file viewing. Your mentor revised that to AI-assisted inspection. We used decoded samples with AI assistance plus CLI/Python measurements. We wrote initial inventories and guesses in DATA.md before building the automated checks and preserved it afterward.

The expected inventory has sixteen streams: four cameras, four calibration streams, six pose/body streams and two magnetic encoders. Calibration describes camera properties; pose gives position/orientation; transforms represent coordinate relationships; encoders measure gripper state. This inventory is specific to the supplied Quest/gripper recording type.

```
wsl -d Ubuntu -- bash -lc "cd /home/haroon/video-qc && make day2-tools"
wsl -d Ubuntu -- bash -lc "cd /home/haroon/video-qc && .cache/bin/mcap info '/mnt/c/Users/win
11/Downloads/data/ep_6fed8bc1.mcap'"
```

Mentor takeaway: understanding the actual recording format first prevents checking the wrong topics, schemas or rates. Selected initial images are not the same as later full automated frame decoding.

8 Stage five ingest by identity and protect raw

Ingestion accepts a recording into the system. The host streams file bytes in chunks and calculates SHA-256. This digest is a sixty-four-character hexadecimal content fingerprint. The worker validates it before uploading. Identical bytes have the same identity even under different names.

Postgres stores one episode per digest. Filename aliases are separate. Reuploading identical bytes adds an alias where needed, rather than a second episode. Renaming the file does not create new evidence, just as photocopying an invoice should not create another purchase.

The MinIO qc-raw bucket stores raw/<digest>.mcap. Conditional creation prevents ordinary replacement. Versioning and COMPLIANCE object retention protect uploaded versions. The exercise retention is 36,500 days, about one hundred years; it is not a production lifecycle recommendation.

Raw immutability protects evidence from pipeline changes. It does not protect the whole laptop from disk loss or cluster deletion. Each supplied recording was verified to have one protected raw object version; no independent disaster-recovery backup was created.

Reports, sampled images, metrics and .rrd viewers are derived data in qc-derived. Their versioned paths contain digest, configuration hash and code hash. A latest convenience key exposes the current result while versioned paths preserve provenance.

State is authoritative information that cannot simply be regenerated: registration, aliases, run history, verdicts and future human review decisions. Derived data can be recreated from raw plus code and configuration. A human decision belongs in durable state, not only in a disposable thumbnail or cache.

Mentor takeaway: identity prevents double-counting, immutable raw preserves the original evidence, and versioned derivation makes recalculation explainable.

9 Stage six orchestrate work with two workers

QcEpisode schedules container inspection, checks for sorted stream IDs, cross-stream checks and final aggregation/publication. It passes identifiers and object references rather than video bytes through workflow history.

Fan-out schedules independent stream checks concurrently. The final deployment has two QC workers and each runs at most one activity at a time. More workers do not guarantee proportional speed because CPU, storage, decompression and database access are shared.

The stable workflow ID contains digest, config version and code version. Submitting a completed episode under the same versions reuses its execution. Failed executions can be resubmitted under the declared reuse policy. A rebuild has its own workflow ID and needs the initial current-version run finalized first.

Activities have a thirty-minute start-to-close timeout per attempt, a fifteen-second heartbeat timeout, and up to five attempts with retry delays starting at two seconds and capped at twenty. These are execution bounds, not promised durations.

A heartbeat reports that work is alive, often with stream progress. If it stops, Temporal can time out and retry the activity. It is currently not a decoder checkpoint: an unfinished stream decodes again from its beginning; completed compatible stream artifacts can be reused.

You chose two workers to reduce memory usage. Each requests 100m CPU and 256 MiB memory and is limited to one core and 768 MiB memory. Its cache uses bounded ephemeral storage. A briefly attempted four-worker setting was restored to two after your response; final manifests and verification use two.

Kubernetes restores the process or pod. Temporal restores the durable work sequence. Idempotent storage operations prevent repeated work from creating repeated logical results. These are three complementary mechanisms.

Mentor takeaway: orchestration, recovery and safe repeated side effects are separate responsibilities.

10 Stage seven check container inventory and timing

Structure and inventory

We validate readability and required header, footer, summary and index. Recoverable truncation means surviving messages can be recovered into a private copy; the original remains unchanged. Unreadable, recoverable-truncated and incomplete readable containers receive distinct reason codes.

Actual topics, schemas and encodings are compared with worker/config/qc.json. Missing required streams, wrong schema and wrong encoding fail. Unexpected streams request review. We count messages, measure durations and compare effective rates with nominal rates.

Timing

log_time is when a message was logged; publish_time is its publication timestamp. Some payloads also have timestamps. We detect duplicate and reversed container timestamps and compare payload stamps against both log and publish time within a 100 ms allowance.

Hz means readings per second. Camera, pose/body and encoder nominal rates are 30, 50 and 100 Hz. At 30 Hz the nominal period is about 33.3 milliseconds. Whole-stream average rate can look healthy despite a local interruption, so gap checks are separate.

A gap greater than three nominal periods is flagged. Dropped frames are estimated from timing rather than observed from an independent source frame counter. Jitter is variation in intervals. p50 is the median, p95 bounds 95 percent of observed intervals, and p99 describes the tail.

Coverage compares starts and ends against the active recording window. Current budgets allow six seconds at startup and 0.5 seconds at the end. The report measures boundary lag and identifies missing time intervals.

Metrics are numeric evidence such as message count or end lag. Reason codes are controlled labels such as GAP_RUN or MISSING_STREAM. A fixed vocabulary avoids arbitrary prose becoming the official decision.

Mentor takeaway: file readability, message continuity and cross-stream coverage are distinct checks.

11 Stage eight decode content and compare cameras

PyAV uses FFmpeg libraries to turn compressed video into images. The pipeline attempts every supplied video frame and counts decode failures. It checks the initial keyframe needed to start decoding dependent compressed frames.

Each decoded frame is resized to 160 pixels wide for low-cost measurements. Downscaling can miss small local defects. We decode and measure all frames, but save JPEG previews and frame-metric rows at about one-second intervals to limit derived storage.

Check	Measurement	Policy
Black or bright run	Sustained mean image intensity	Failure
Frozen run	Very small consecutive-image difference	Review
Blur run	Low Laplacian variance sharpness score	Review
Resolution change	Image width or height changes	Failure
Decode and initial keyframe	Decoder errors or invalid starting dependency	Failure
Camera alignment	Nearest-time skew and changing drift	Failure beyond limits
Sensor validity	Nonfinite numbers or encoder stillness	Nonfinite fails; stillness reviewed

A run is sustained evidence, not one isolated image. Dark/bright runs use half a second; blur and apparent freezing use two seconds. A legitimate dark scene, stationary camera or low-texture view can trigger a heuristic.

Skew is a camera time offset. Drift is how alignment changes during the episode. Current camera budgets are 50 ms skew and 30 ms drift. Timestamp alignment does not prove every visual event matches across cameras.

Sensor checks cover timing, coverage, nonfinite numbers such as NaN/infinity and encoder flatlines. We did not implement a general robot-motion model proving sensor values agree with visible motion, every sensor saturation pattern, or every possible video-format change.

Mentor takeaway: we implemented more than the minimum tier-three checks, with a declared scope and scene-dependent limitations.

12 Stage nine choose defensible parameters

Your mentor asked you to make the choices. We removed the blanket mentor-confirmation gate and record parameter_authority as engineer_selected. Parameters live in a versioned configuration, and reports identify the exact software and rules used.

Parameter	Setting	Basis or limitation
Rates	30 / 50 / 100 Hz	Observed stream families, not independently certified specifications
Rate tolerance	30 percent	Allows variability; gaps are checked separately
Gap	More than 3 periods	Detect sustained interruptions
Coverage start / end	6 / 0.5 seconds	Observed startup up to 5.27 s; camera end lag under 0.08 s
Payload timestamp mismatch	100 ms	Initial alignment budget
Camera skew / drift	50 / 30 ms	Initial synchronization budgets
Dark / bright mean	Below 10 / above 245	Scale 0 black to 255 white
Brightness run	0.5 seconds	Require sustained extremes
Freeze difference / run	At most 0.2 / 2 seconds	Static scenes can be legitimate
Blur score / run	Below 10 / 2 seconds	Texture and resizing affect sharpness
Encoder flatline	5 seconds	Unmoving grippers can be legitimate
Stored image cadence	About 1 second	Storage tradeoff; decoding remains full

Initially both boundaries allowed six seconds. You approved tightening only the end to half a second. All 164 supplied camera streams had maximum end lag about 0.0797 seconds. A two-second missing-tail fixture proves the new policy detects a defect hidden by the old allowance.

A defendable threshold has a purpose, a unit, observed support, a test near its boundary and a stated limitation. It is not perfect because it is precise. Changes should have documented reasons, not simply make more files pass.

Mentor takeaway: we own the standard and can explain it, while reserving ambiguous evidence for review and acknowledging calibration limits.

13 Stage ten store decisions and metrics safely

Postgres is the operational source of truth. qc_episodes registers identity, qc_aliases stores names, qc_runs preserves versioned configurations/results, qc_verdicts holds one current verdict per digest, and qc_deliveries records publication bookkeeping.

Uniqueness constraints prevent duplicate episode and current-verdict identities. An upsert inserts if absent or updates the existing identity. A transaction commits a set of database changes together or rolls them back. Small connection pools reuse a bounded set of database connections.

ClickHouse is a rebuildable analytical projection of measurements. Postgres suits transactions and relational integrity; ClickHouse suits large numerical scans. Putting all frame metrics beside operational state would add indexing/write load; treating ClickHouse as verdict authority complicates transactional uniqueness.

Idempotence means repeating an operation preserves the intended final effect. Stable artifact keys, delivery IDs and metric row IDs, missing-row checks, deduplication tokens and database upserts prevent duplicate logical publication under the tested constraints.

Reports and metrics are published before committing the official verdict. This is not one distributed transaction spanning all three stores. Partial external publication is recovered through repeatable writes and repair rather than a universal atomic commit.

Rerun .rrd artifacts contain sampled images and typed findings on episode_seconds. Native viewer verification checks file validity, not whether the interpretation is correct. The viewer is not full original-video playback.

The fleet query asks which camera has most estimated drops across the 41 mentor recordings. Synthetic fixtures are excluded. All four cameras tied at zero estimated drops under our model, and the query matched report totals.

Mentor takeaway: current operational truth and derived analytics have different roles; exactly-one logical outcomes require explicit identity controls.

14 Stage eleven prove crash recovery and rebuilding

Pilot duplicate tests preserved one episode, one raw version and one current verdict. We also checked metric identities, viewer validity and workflow replay.

You approved deletion of one busy worker pod. Kubernetes replaced it and the workflow completed, but the activity finished during graceful shutdown. This proved pod replacement, not a genuine interrupted-task retry. We reported that distinction.

You separately approved one worker-process crash. We identified an exact busy QC container and sent SIGKILL through the node runtime. The container exited 137 and restarted once. Temporal recorded a heartbeat timeout and attempt two of the same activity.

The historical ep_23f5a0d2 execution completed with one episode, one verdict, one raw version and 2,501 unique metrics. The tested execution took about 96 seconds. That evidence records its older code/config versions; it is not a fresh crash test of every later change.

Replay reconstructs workflow scheduling from history. Rebuild reads raw, reruns checks and republishes projections. A representative final-policy rebuild preserved one verdict and 1,275 distinct metric rows, and passed viewer verification and rebuild replay.

For a coverage-only policy change, we reused compatible measurements after comparing exact function/library fingerprints and unchanged non-coverage settings. Each stream records source provenance. Container inspection, cross-stream checks and publication ran again, followed by a full raw-based rebuild consistency check.

This optimization did not achieve the stretch goal of re-QC entirely from ClickHouse without raw reads. Container inspection still reads raw and prior stream artifacts supply the reused measurements.

Mentor takeaway: replacement, genuine retry, replay and full rebuild are different claims. Both destructive approvals were consumed; another live deletion or kill needs fresh approval.

15 Stage twelve create controlled damage tests

Because there is no independent real-recording answer key, we made controlled test copies under the ignored repo cache. A fixture has a deliberate, documented defect. Original source hashes were checked and source bytes remained unchanged.

The final suite has nineteen cases: one expected pass, fifteen failures and three reviews. All matched their expected verdict and required reason codes. Related secondary findings are allowed: a uniform black image may also look frozen or blurred.

- Container: unreadable bytes, recoverable truncation, unindexed content, and a complete file with no summary.

- Inventory: missing camera, wrong schema, unknown encoding and unexpected stream.

- Timing: duplicate/reversed timestamps, a gap, early ending, a two-second missing tail, and payload versus publish timestamp mismatch.

- Content: undecodable video, black, frozen and blurred video, plus a clean control.

Testing found real bugs

- Payload stamps initially compared only with log_time; publish_time comparison was added.

- Complete but summary-free content initially looked like recoverable truncation; it now uses CONTAINER_INCOMPLETE.

- CLI programs needed to close the Postgres pool cleanly to avoid interpreter shutdown trouble.

- Rerun flush used an unsupported argument and was fixed.

- Verification assumed timing fields existed even for empty/damaged streams; safe handling was added.

- Projection repair needed to account for missing rows despite previous deduplication records.

- Fixture creation once overlapped ingestion; hash validation rejected the changing copy, and generation/upload became sequential.

A confusion table compares expected classes with predictions. Its diagonal contains correct matches: one pass, fifteen fail and three review here. Nineteen out of nineteen is controlled-case agreement, not 100 percent accuracy on every future recording.

Mentor takeaway: tests deliberately challenged assumptions and found defects in our own implementation.

16 Stage thirteen verify the full dataset

Verification area	Recorded result
Supplied recordings	41 current reports: 29 pass, 1 fail, 11 review
Identity and protection	Original digests match; one protected raw version each
Operational decisions	One current authoritative verdict each
Video checks	All supplied frames decoded; no unavailable checks
Viewer artifacts	All 41 passed native verification
Temporal histories	All 41 passed replay
Metric projection	No duplicate logical row identities
Dataset fleet query	Report match; four-way tie at zero estimated camera drops
Controlled cases	19 expected verdict/reason matches

ep_346eb2d7 failed for BLACK_RUN across all four cameras around 21.76 to 23.12 seconds, with small stream differences. It failed our brightness standard. We did not independently establish whether the cause was a covered lens, dark environment or recording fault.

All eleven review cases have SENSOR_FLATLINE. They request context rather than representing eleven broken workflows. If a downstream team requires a binary shipping decision, these need documented human interpretation.

The first-read DATA.md guesses agreed with 26 of 41 final policy verdicts. Eleven initially likely-good files now request review; four initially uncertain files passed. This is preliminary agreement, not measured accuracy.

The exercise mentioned a mentor answer key; you clarified none exists. We prepared a reference-review worksheet and scoring tool for future documented independent labels rather than inventing truth. We can defend policy and controlled tests, but cannot claim independently measured real-data accuracy.

We wrote the acceptance standard, short design note, operating guide, final report and twenty-minute demo. The actual mentor presentation has not been performed. A fresh live worker kill in that demo still needs approval.

Mentor takeaway: technical deliverables are verified within their scope, while human review and accuracy claims remain separate.

17 Stage fourteen add the local web dashboard

The dashboard lives under web/ and runs at http://localhost:8090. make web starts a Python server on the WSL host. It is not deployed in Kubernetes yet, and make up does not start it automatically.

HTML supplies structure, CSS appearance and JavaScript behavior. The Python backend exposes APIs returning JSON. The browser talks to that backend; it calls the existing worker through kubectl. Storage credentials remain inside the worker, not in the browser.

You can search, filter verdicts, choose the mentor dataset or all uploads/tests, inspect real findings, enter an episode time, load nearest saved camera samples and download full JSON. The library refreshes about every fifteen seconds. Pending records can show available workflow state; this is not a detailed per-activity console.

Uploads accept one MCAP up to 8 GiB at a time. Bytes stage under .cache/web, then use the existing hash-verified immutable ingest and workflow submission. Available staging disk is checked. Transfer progress, storage progress, submission and final verdict are different events. Identical uploads reuse identity.

The server binds to loopback 127.0.0.1 and checks expected Host values, rejects cross-origin uploads and requires an upload header. Only defined routes are served. This is a local unauthenticated tool, not a public multi-user production website.

Tests verified live totals, exported-versus-live report equality, four JPEG samples, invalid route/host/origin/file rejection, and duplicate ingestion of a damaged fixture and a valid 15 MB control. Browser checks verified filtering, report panels and actual images.

The web stage is commit 6f1666c, local-only at this checkpoint. The pipeline commits had been pushed earlier. Upload activity is session-local; restart clears those messages but does not erase persistent results. Staging files remain and use disk.

Full playback, saved human decisions, detailed progress/retry controls and cluster packaging remain. Mentor takeaway: the dashboard presents the pipeline evidence; it does not replace the checking worker.

18 How to read a report

JSON stores named values. Braces group an object, brackets form a list, [] is an empty list and null means absent or not applicable. Read decision, reason, affected stream, interval, measurements and rule in that order.

Field	Question
verdict	What decision was made?
reason_codes	Which fixed labels explain it?
findings	Where and when did the condition occur?
streams	What message, decoding and timing metrics support it?
cross	Do coverage and camera times align?
config	Which thresholds and expected inventory applied?
digest	Which exact bytes were checked?
config_version / code_version	Which rules and software produced it?
measurement_source	Which compatible earlier measurement versions were reused?

```
code: BLACK_RUN
stream: /quest3s/head_left/camera/compressed
start_s: 21.759965454
end_s: 23.119963545
observations: 42
```

This means 42 observations formed a dark run lasting about 1.36 seconds. Compare that with the half-second duration rule and mean brightness threshold 10 on the 0 to 255 scale. Load images near 22.44 seconds and read actual sample timestamps. A nearest sample is not every image in the interval.

ep_0ed34c0f has right-encoder flatlines around 94.97-102.19 and 139.12-146.03 seconds. Both exceed five seconds. Compare visible gripper behavior with readings; stillness can be legitimate. Automated flatline detection does not itself prove a motion contradiction.

ep_23f5a0d2 passes. Its head-left camera has 3,828 messages and decoded frames, zero decode errors, about 30 Hz, no duplicate/reversed timestamps and zero estimated drops. These support the result within policy. A calibration stream can correctly have zero decoded frames because it is not video.

Do not judge a number without its unit and stream kind. seconds and milliseconds differ by a thousand. end_lag_s means missing tail relative to the active window; skew_max_ms measures camera alignment. An empty findings list means no implemented rule triggered, not that every possible defect was excluded.

19 Commands to start and operate now

All commands below are for PowerShell and run inside WSL. Copy full lines from the HTML companion. Keep web and UI terminals open; stop those host services with Ctrl+C instead of destroying the cluster.

Start or reuse infrastructure

```
wsl -d Ubuntu -- bash -lc "cd /home/haroon/video-qc && make up"
```

Requires Docker access and the tools documented in README. Uncached images/source builds need network access, sufficient memory and disk. First startup may be much slower than cached reuse. It starts cluster services, not the host web dashboard.

Check readiness

```
wsl -d Ubuntu -- bash -lc "cd /home/haroon/video-qc && make status"
wsl -d Ubuntu -- bash -lc "cd /home/haroon/video-qc && make verify"
```

Services should be Ready, while successful bootstrap jobs are Completed. Twelve Ready service pods and two Completed jobs were last verified for this expanded stack; changes can alter counts.

Start the recording dashboard

```
wsl -d Ubuntu -- bash -lc "cd /home/haroon/video-qc && make web"
```

Open http://localhost:8090. Keep this terminal running. Ctrl+C stops the dashboard only; it leaves the cluster and saved results running.

Open Temporal in another terminal

```
wsl -d Ubuntu -- bash -lc "cd /home/haroon/video-qc && make ui"
```

Open http://localhost:8080 and Temporal namespace default. This is processing history, not full video playback. Ctrl+C closes the port-forward.

Run the infrastructure example

```
wsl -d Ubuntu -- bash -lc "cd /home/haroon/video-qc && make hello"
```

Expect Hello, world! and a completed execution. This confirms the small workflow path, not all recording checks.

20 Commands to submit inspect and verify

Submit one original and wait

```
wsl -d Ubuntu -- bash -lc "cd /home/haroon/video-qc && make qc-submit FILE='/mnt/c/Users/win
11/Downloads/data/ep_0ed34c0f.mcap'"
```

Quoting preserves the space in win 11. A folder path submits its MCAPs. Repeating identical bytes under the same versions is safe. The output includes the full digest.

List episodes and download artifacts

```
wsl -d Ubuntu -- bash -lc "cd /home/haroon/video-qc && make qc-status"
wsl -d Ubuntu -- bash -lc "cd /home/haroon/video-qc && make qc-report DIGEST=YOUR_DIGEST"
wsl -d Ubuntu -- bash -lc "cd /home/haroon/video-qc && make qc-viewer DIGEST=YOUR_DIGEST"
```

Replace YOUR_DIGEST with the complete 64-character fingerprint. qc-status includes synthetic fixtures. Downloads appear under .cache/qc-exports. JSON needs a text editor; .rrd needs a compatible Rerun viewer. Downloading does not install that viewer.

Verify the dataset and fleet

```
wsl -d Ubuntu -- bash -lc "cd /home/haroon/video-qc && make qc-verify-dataset"
wsl -d Ubuntu -- bash -lc "cd /home/haroon/video-qc && make qc-fleet"
```

Dataset verification checks saved current outputs, raw protection, metric identities, viewers and replay and refreshes local evidence. It does not re-decode every recording. The fleet query excludes synthetic cases.

Strict check for a clean completed recording

```
wsl -d Ubuntu -- bash -lc "cd /home/haroon/video-qc && make qc-verify DIGEST=YOUR_DIGEST"
```

This strict verifier requires complete decoding and available checks. A corrupted recording may correctly fail the strict verifier even though its quality fail verdict is right. Use controlled/full-dataset verification plus its report for damaged inputs.

Compare early guesses and verify the web app

```
wsl -d Ubuntu -- bash -lc "cd /home/haroon/video-qc && make qc-compare"
wsl -d Ubuntu -- bash -lc "cd /home/haroon/video-qc && python3 web/verify.py"
```

Keep make web running for the web test. It resubmits an existing damaged fixture when available and writes verification/web.json. qc-compare measures agreement with preliminary guesses, not independent accuracy.

21 Maintenance commands and destructive boundaries

Inspect resources without changing them

```
wsl -d Ubuntu -- bash -lc "cd /home/haroon/video-qc && source scripts/tool-env.sh && kubectl
--context kind-qc -n qc get pods,pvc"
wsl -d Ubuntu -- bash -lc "cd /home/haroon/video-qc && source scripts/tool-env.sh && kubectl
--context kind-qc -n qc describe pod minio-0"
```

tool-env.sh selects the repo-pinned tools. Use actual generated worker pod names from get pods. describe reveals events and errors. Do not print Secrets, environment credential values, tokens or credential files.

Rebuild from protected raw

```
wsl -d Ubuntu -- bash -lc "cd /home/haroon/video-qc && source scripts/tool-env.sh && kubectl
--context kind-qc -n qc exec deployment/qc-worker -- python -m qc.cli rebuild YOUR_DIGEST --wait"
```

Use only after initial current-version finalization. Rebuild reruns checks and repairs/overwrites derived outputs while preserving logical identities. It costs more than verification of saved outputs.

Create stable test copies then submit

```
wsl -d Ubuntu -- bash -lc "cd /home/haroon/video-qc && make day2-tools"
wsl -d Ubuntu -- bash -lc "cd /home/haroon/video-qc && make qc-damage-fixtures
FILE='/mnt/c/Users/win 11/Downloads/data/ep_07e258d4.mcap'"
wsl -d Ubuntu -- bash -lc "cd /home/haroon/video-qc && make qc-test-damage"
```

Finish fixture generation before ingestion. These create derived damaged copies and test episodes; originals remain untouched. They are not destructive worker-crash tests.

Inspect local Git history

```
wsl -d Ubuntu -- bash -lc "cd /home/haroon/video-qc && git status --short && git log --oneline -8"
```

Review selected code/docs/evidence before committing. Raw files and caches stay ignored. Commit is local; push publishes to GitHub. Do not blindly add unexpected files.

Teardown only when losing all local cluster data is acceptable

```
wsl -d Ubuntu -- bash -lc "cd /home/haroon/video-qc && make down"
```

It asks you to type qc and destroys cluster volumes, object storage and workflow history. It is not how you close a webpage. A new pod deletion or worker kill needs fresh explicit approval; both prior one-time approvals have been used.

22 Troubleshooting in plain language

8080 will not open

Check make status, start the cluster if needed, then run make ui in a separate terminal and leave it open. Read port-forward errors. Port 8080 is Temporal and 8090 is our dashboard.

8090 will not open

Run make web and leave its terminal open. Address already in use may mean an existing server is there; try that page first. Do not kill an unidentified process. A loaded page with failed results can mean the host server works but the cluster worker is unavailable.

Pod is Running but not Ready

The process started but readiness has not passed. Describe the pod and inspect events. Initialization can take time; repeated redeploys without diagnosis do not establish a fix.

ImagePullBackOff

Read the actual describe events. A tag may be missing, access denied, rate limited or unreachable. Verify pinned tags rather than changing to latest. Our source-built MinIO and worker images need loading into kind.

Upload submitted but no verdict

Select All uploads + test fixtures and wait for refresh. Inspect the matching Temporal workflow and worker readiness. Submission is not completion. Unknown state means unavailable lookup, not definitely running.

Temporal Completed but report fail

This can be correct: the job completed and found bad quality. Read the reasons. CHECK_UNAVAILABLE means required evidence could not be checked; fail closed avoids approving it.

A review finding looks harmless

Inspect the interval and recording context. Stillness or low texture may be legitimate. Document your reference interpretation without changing the machine verdict merely to agree. The UI does not yet save manual review.

Disk grows or processing is slow

Originals, staging, worker caches, Docker caches, viewers and databases use different storage. Staging is not automatically deleted. Diagnose within scope before approved cleanup. Full decoding and uncached source builds are costly. Do not use prune or cluster deletion as an unexplained shortcut.

23 Mentor questions about infrastructure

Why Kubernetes on one laptop?

The exercise teaches deployments, stateful services, resource limits, readiness and replacement. kind provides a local cluster. It is a learning architecture, not a claim that every small script needs Kubernetes.

Docker versus Kubernetes?

Docker packages and runs containers. Kubernetes schedules and maintains workloads across nodes. kind creates local nodes using Docker.

Deployment versus StatefulSet?

Deployments maintain replaceable worker replicas. StatefulSets maintain stable identities and storage association for our databases and MinIO.

Why Helm?

The official Temporal chart supplies a maintained multi-service installation. Our values file controls connections and resources, and versions are pinned.

Why source-build MinIO?

The attempted community binary image path could not be pulled. You approved pinned official source with a checked commit and local kind loading.

What does PVC storage guarantee?

Persistence across individual pod replacement in this cluster, not survival of cluster deletion or laptop failure.

Why two workers?

You chose the lower memory footprint. More concurrency needs evidence about shared CPU, storage and service capacity.

Why Completed bootstrap pods?

They are successful one-time jobs. Services should remain Ready; jobs should finish.

Where are credentials?

Current values stay in Kubernetes Secrets and containers. The browser never receives them. Old pre-cleanup development credentials remain in earlier Git history; recreated services use new values and we do not reproduce credentials here.

Was fresh startup tested?

Yes, after approved teardown from a fresh clone with no existing qc cluster or volumes, reusing Docker caches. Later startup tests used the populated cluster, a different scope.

24 Mentor questions about reliability

Why digest instead of filename?

Identical bytes can have different names and one name can represent different bytes. Digest identifies contents; aliases retain the names.

What is immutable?

Pipeline-uploaded raw is not modified, moved or replaced. Conditional creation and COMPLIANCE retention protect it, but deleting the entire local cluster can destroy storage.

What is idempotence?

Repeating an operation produces the same intended final effect. Identity constraints, stable workflow/artifact/metric IDs and upserts prevent logical duplicates.

What happens when a worker dies?

Heartbeats stop, Temporal times out and retries the activity, Kubernetes restores the container/pod, and repeatable publication avoids duplicate final results.

Does heartbeat resume the last video frame?

No. It reports liveness/progress. An unfinished stream restarts decoding from its beginning.

Does replay decode again?

No. It reconstructs workflow scheduling from recorded events/results. A raw-based rebuild repeats checks.

Why separate Postgres and ClickHouse?

Postgres owns transactional state and current verdicts. ClickHouse is a rebuildable analytical measurement projection.

Are all stores committed atomically?

No distributed transaction spans them. We publish stable external outputs and then commit the verdict, with retry/repair for partial work.

How was rebuilding proved?

A representative final-policy raw-based rebuild retained one verdict and 1,275 unique metric rows and passed viewer verification and replay.

Did re-QC use only ClickHouse?

No. Coverage-only reuse used fingerprinted earlier stream artifacts; container inspection still read raw. Pure metric-only rethresholding remains a stretch goal.

25 Mentor questions about quality and the UI

Why is flatline only review?

An unchanged encoder can mean a stationary gripper. Apparent freezing or blur can also be legitimate. The finding is evidence requiring context.

Are your standards perfect?

No. They are documented choices with observed support, controlled tests and explicit limits. Defensible does not mean universally accurate.

What accuracy can you claim?

Nineteen controlled cases matched expectations. Real-data accuracy is not independently measured. Twenty-six of forty-one agreement with initial guesses is not an answer-key score.

Why tighten end coverage only?

Startup delay reached about 5.27 s, but camera end lag was below 0.08 s. A six-second end budget hid missing tails, so 0.5 s is better supported and tested with a two-second defect.

Does zero estimated drops prove no loss?

No. It is the outcome of our chosen timing estimator, without an independent source frame counter. Content defects can exist with healthy timing.

Were all images inspected?

Initial AI-assisted observations used selected decoded images. Automated QC later decoded and measured every supplied frame. Stored previews remain about one second apart.

What is fail closed?

An unavailable required check prevents pass. A decoder failure or unknown encoding never implies healthy data.

Is the website production ready?

It is a local loopback tool without public hosting or multi-user authentication. It supports first-stage uploads and evidence inspection.

What breaks at 500 workers?

Repeated scans, bandwidth, decompression, unbounded cache growth, small writes, connections and bursts. Improvements need bounded cache, backpressure, queue separation, batching, connection budgets and version routing.

What remains?

Review interpretations, independent labels if accuracy is needed, the actual mentor demo, and optional web playback, saved human decisions, richer progress/retry and cluster packaging.

26 Files and pinned versions to recognize

Location	Purpose
Makefile and scripts/	Start, deploy, operate and verify
k8s/ and images/minio/	Manifests, Helm values and upstream MinIO build
worker/hello/ and worker/qc/	Example and real workflow/checks/storage/CLI
worker/config/qc.json	Inventory, thresholds and verdict policy
worker/sql/	Operational tables and metrics schema
tools/	Host submit/export, fixtures, fleet and verification
DATA.md and data/	Frozen first read and observations
reports/ and verification/	Exported outcomes and evidence
web/	Local server, browser UI and smoke tests
docs/	Design, standard, operations, demo and guide
.cache/ and .venv/	Ignored staging, tools, raw copies and inspection libraries

Component	Recorded pin
Kubernetes node	kindest/node v1.30.0 with image digest
Postgres / ClickHouse	16.4 / 25.8.4.13
Temporal chart	1.7.0
Temporal server and tools / UI	1.32.0 / 2.54.1
MinIO upstream release	RELEASE.2025-10-15T17-29-55Z
MinIO source commit	9e49d5e7a648f00e26f2246f4dc28e6b07f8c84a
QC image / Python base	video-qc/qc-worker:pipeline-v1 / 3.12.12-slim-bookworm
Python Temporal SDK / MCAP CLI	1.14.1 / 0.3.0
Exercised local tools	kind 0.23.0; Helm 3.22.0; Docker 29.9.0; Make 4.4.1

Tags name versions; digests identify exact image content; chart versions pin templates. README and Dockerfiles record full digest pins. Worker requirement files pin library versions. Day Two exploratory libraries are a separate environment and can differ.

The local pipeline-v1 tag is rebuilt from current worker source. A code hash records actual worker code and a deployment annotation triggers rollout. A local tag alone is not complete provenance, and a Git commit ID is not the same as a worker code hash.

27 Remaining work and your practice session

- Complete: infrastructure, hello workflow, empty-cluster proof, frozen first read, protected ingestion, two-worker QC, content/timing checks, all 41 reports, viewers, fleet query, controlled tests, replay, historical genuine crash retry and representative rebuild.

- Complete: acceptance standard, design note, operating guide, demo materials and first local dashboard.

- Remaining human work: interpret review cases as needed, collect independent labels if accuracy is required, and present the mentor demonstration.

- Remaining optional product work: full playback/timeline, saved human reviews, rich progress/retries and dashboard deployment through make up.

verification/reference-review.csv provides a reference worksheet, and tools/score_reference.py scores documented labels with reviewer/evidence. These are not an external answer key or a UI-backed review database. Human overrides should be stored separately from the machine verdict.

Future changes should explain why, record versions and revalidate affected behavior. Avoid changing worker code during active runs without draining or activity-version routing. Keep raw immutable, DATA.md frozen and intentional damage fixtures separate.

A practical learning exercise

Run status and verify. Start the dashboard. Inspect ep_346eb2d7 at about 22.44 seconds and explain BLACK_RUN plus the rule's limitation. Inspect ep_0ed34c0f and explain why unchanged readings need context. Open Temporal and explain why Completed does not mean pass.

You understand the project when you can follow one file through fingerprint, raw storage, workflow, activities, metrics, verdict and displayed evidence, and can state exactly what each verification proves.

28 Evidence and keeping this guide current

This guide describes the checkpoint on 14 October 2026 using the supplied exercise, your later directions, repository code, Git history through 6f1666c and saved verification. It does not guarantee that services stay running after terminals close or the laptop restarts.

- README.md: prerequisites, versions and startup scope.

- REPORT.md and docs/ACCEPTANCE-STANDARD.md: results and policy limitations.

- docs/DESIGN.md and docs/OPERATING.md: authority, retries, rebuild and commands.

- verification/day1.json: approved teardown, empty-cluster startup and hello completion.

- verification/pipeline-crash.json: exact historical crash/retry and tested versions.

- verification/damage-fixtures.json: controlled expectations and results.

- verification/full-dataset.json and reports/: all supplied results and verification.

- verification/final-rebuild.json: final-policy representative rebuild.

- verification/web.json and docs/WEB.md: local dashboard tests and limits.

- docs/DEMO.md: twenty-minute presentation plan.

Some older notes still mention mentor confirmation or an answer key, and REPORT.md predates the dashboard. Your later directions and the current acceptance policy supersede those expectations. The exercise's mentor-only preparation notes are background, not permission to alter original data.

Older Git history includes pre-exercise development credentials. We do not print their values or rewrite history as part of this guide. Any future history cleanup is a separate reviewed action.

If code, thresholds, service versions or replicas change, update the guide and verification with honest dates and scope. Do not relabel an old test as a new one. The HTML companion contains copyable command lines; the PDF is the reading edition.